NICS 清華大學

Tsinghua University

Modern Computer Architecture
Fall 2025

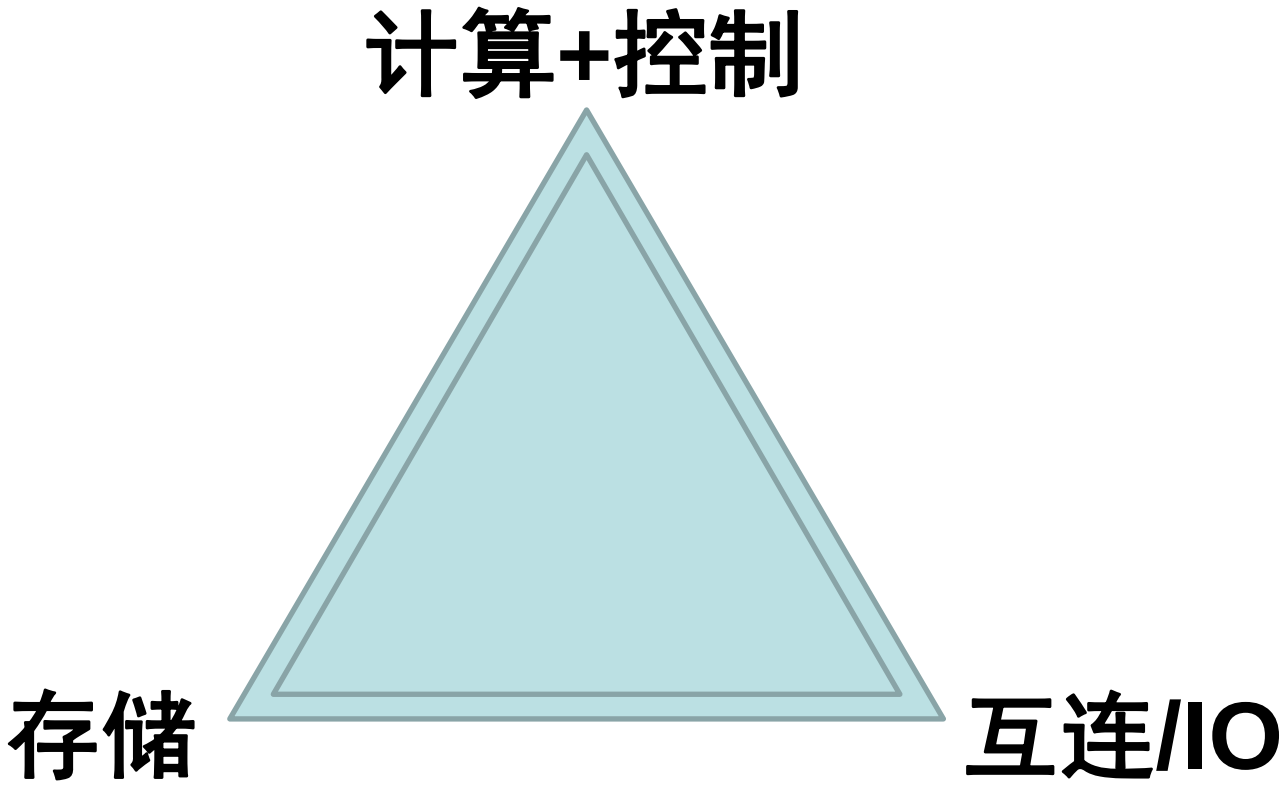
Lecture 07_1: Memory Hierarchy

Yongpan Liu

ypliu@tsinghua.edu.cn

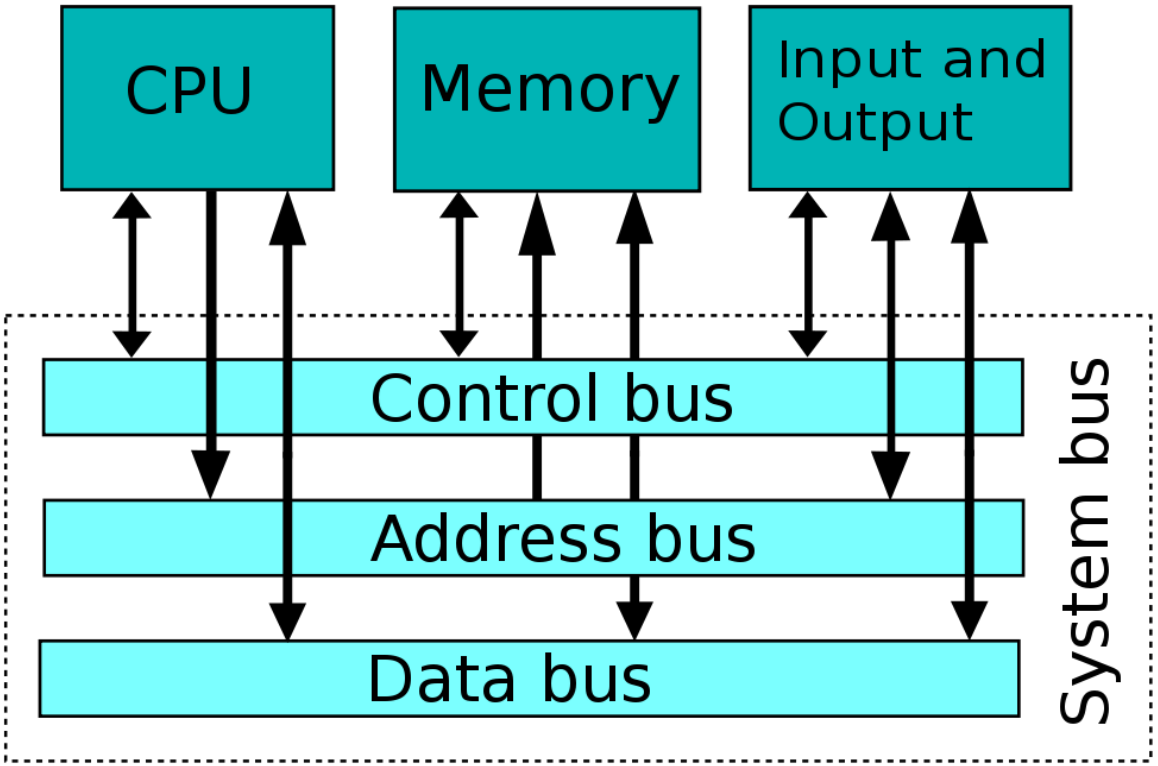
2025-Nov.-12

Computer Architecture

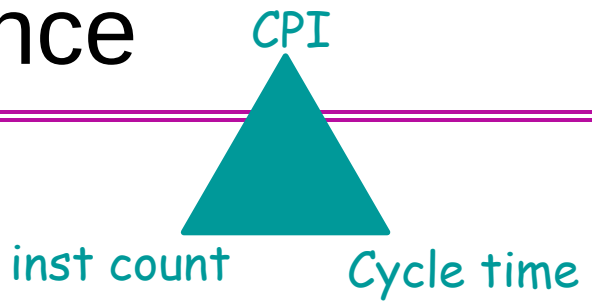


Computer Architecture

- **Processor**
- **Memory**
- **Interconnection/Devices**
- **Software**



Computer Performance



$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

	Inst Count	CPI	Clock Rate	
Program	X			程序
Compiler	X	(X)		
Inst. Set.	X	(X)		架构
Organization		X	X	
Technology			X	电路

After pipeline, What if I want better performance?

- CPU Time = IC × CPI × CCT

- The best CPI we can achieve so far:

  CPI = 1

  CPI<1?

- The best CCT we can achieve ?

What if we want better performance?

Extracting Yet *More* Performance

- Two options:
  - Lower CCT / Higher CR :: superpipelining

    - Increase the depth of the pipeline to increase the clock rate

  - Lower CPI / Higher IPC :: multiple-issue :: parallel

    - Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions)

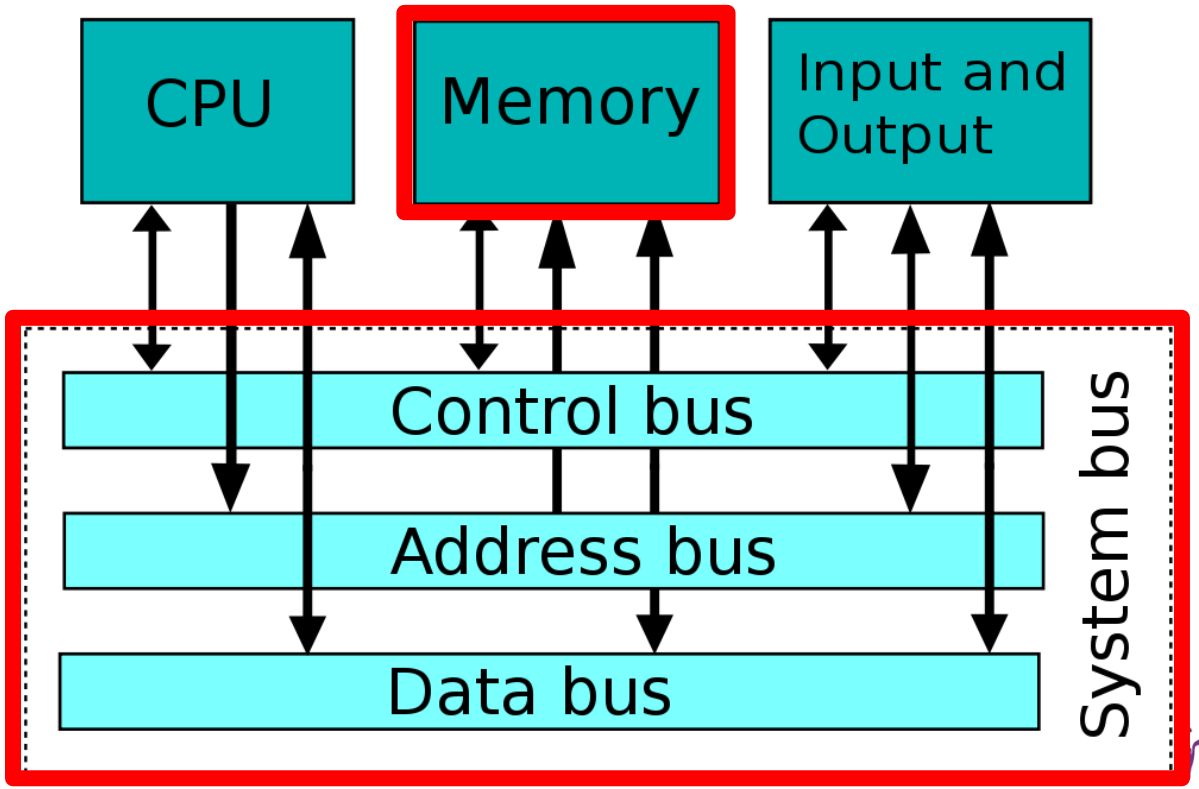
$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

Extracting Yet *More* Performance

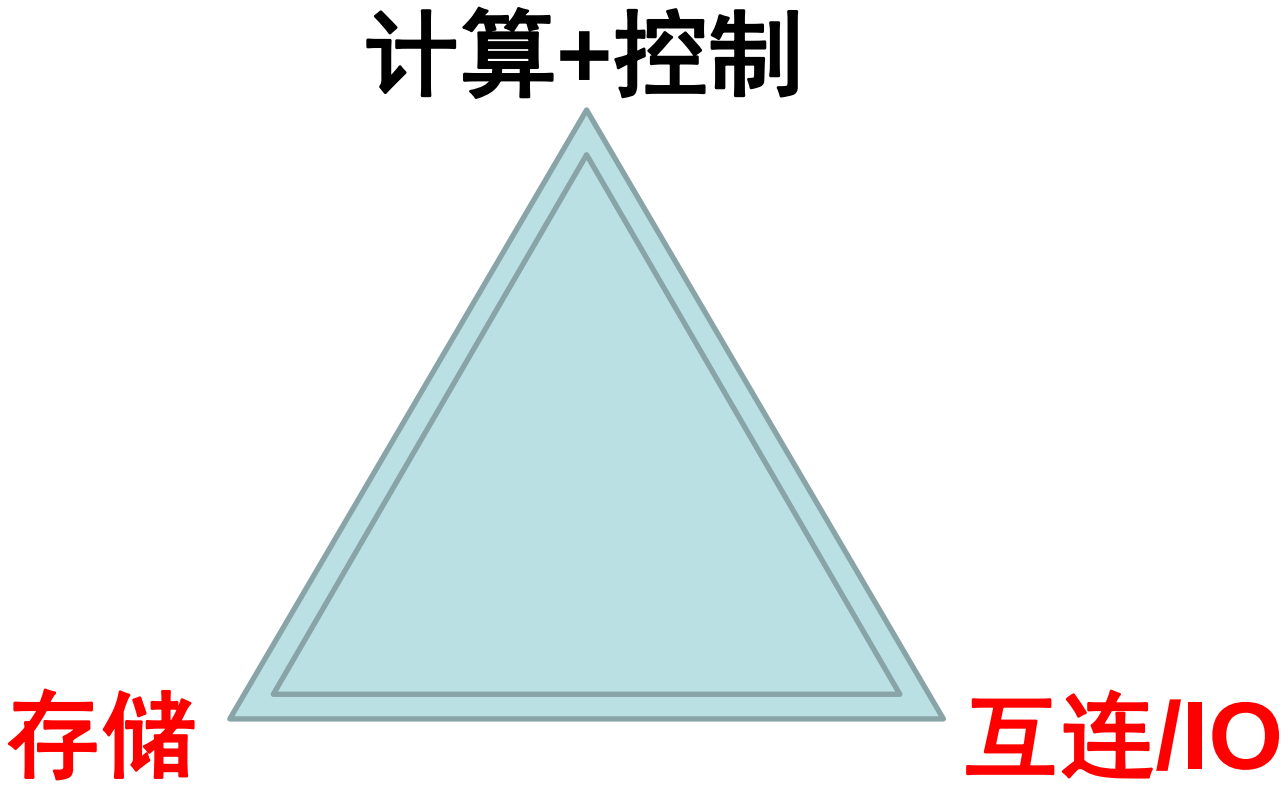
- Two options:
  - Increase the depth of the pipeline to increase the clock rate – superpipelining（流水线深度）
  - Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions) – multiple-issue（流水线宽度）
- Launching multiple instructions per stage allows the instruction execution rate, CPI, to be less than 1
  - So instead we use IPC: instructions per clock cycle
    - E.g., a 6 GHz, four-way multiple-issue processor can execute at a peak rate of 24 billion instructions per second with a best case CPI of 0.25 or a best case IPC of 4
  - If the datapath has a five stage pipeline, how many instructions are active in the pipeline at any given time?
    - 5 -> 20

Performance Bottleneck

- **Processor**
- **Memory**
- **Interconnection/Devices**
- **Software**



Computer Architecture



Outline of following 3 weeks

- Memory Parts:
  - Memory Overview
  - Memory Hierarchy
  - Cache Intro.
  - Improving Cache Performance
  - Virtual Memory
  - Disk & RAIDs
  - Summary of Memory
- Future Computing Arch for PIM & CIM
- Interconnect/IO Parts:
  - Bus & I/O System

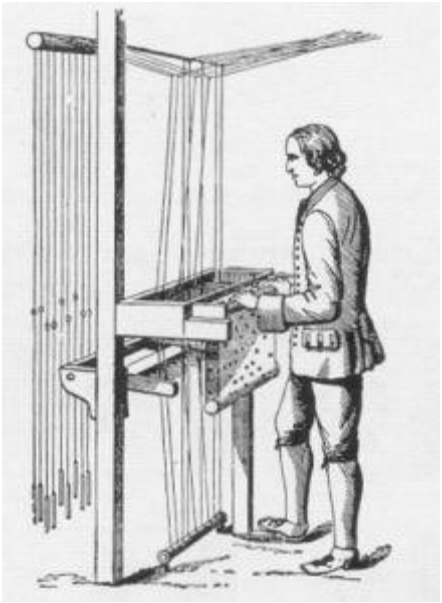
Outline of Today

- Memory Overview
- Memory Hierarchy
- Cache Introduction

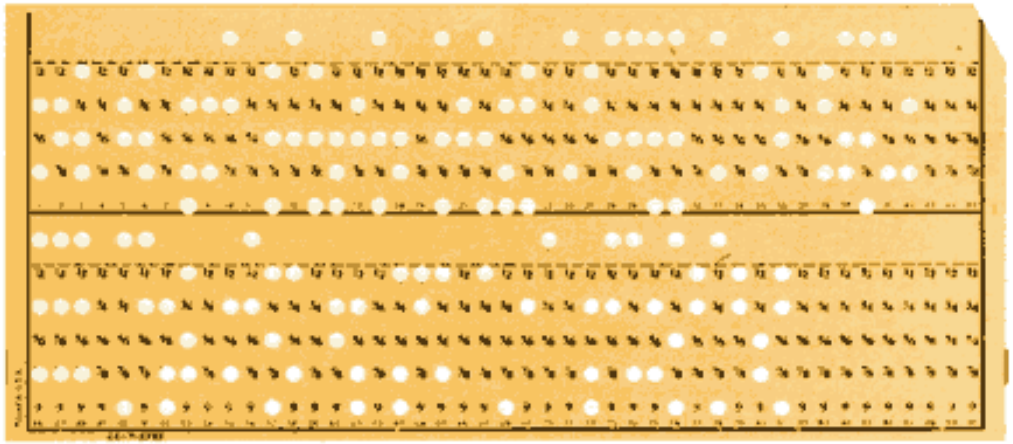
Memory History

- 打孔卡纸和穿孔纸带
- 电子管计算机时期(1946 ~ 1959)
  - 主存储器：水银延迟线存储器、计数电子管、磁鼓和磁芯存储器，磁带
- 晶体管计算机时期(1959 ~ 1964)
  - 主存储器：磁芯存储器
  - 辅助存储器：磁鼓和磁盘
- 集成电路计算机时期
  - 主存储器：半导体存储器
  - 辅助存储器：磁盘，普遍采用虚拟存储技术，光盘

打孔卡纸和穿孔纸带



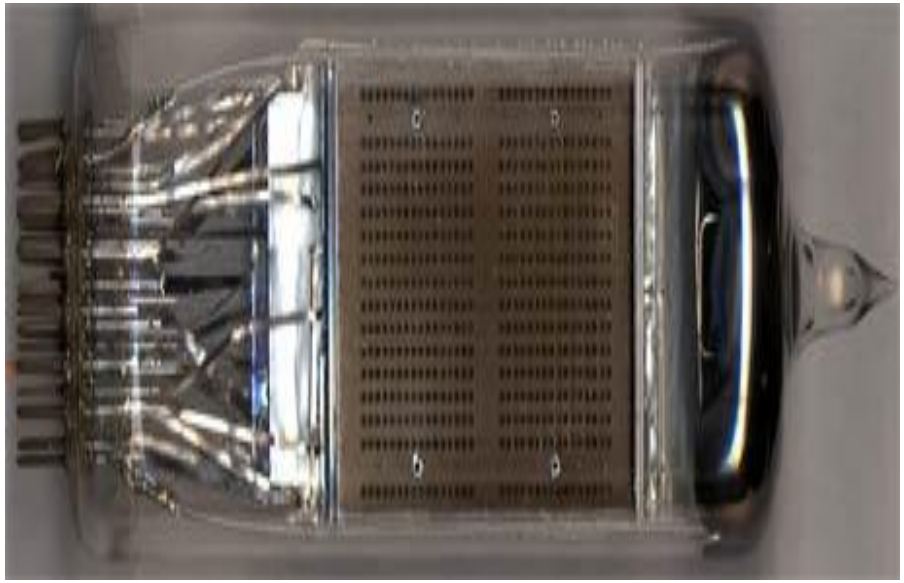
1725年：法国纺织机械师布乔（B. Bouchon）发明了“打孔卡纸”的构想。



90列孔

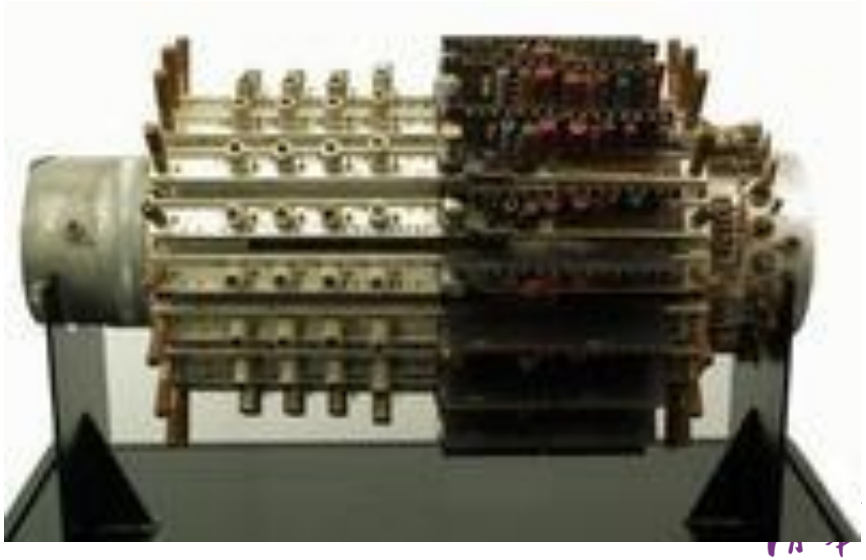
Alexander Bain（传真机和电传电报机的发明人）在1846年最早使用了穿孔纸带。纸带上每一行代表一个字符，显然穿孔纸带的容量比打孔纸卡大多了。

计数电子管和汞延时线



1946，计数电子管
25厘米长，4096bits

1951，汞延时线40℃
直径10mm、长150cm的管子，
内部有很多充满水银的管道，
使每个汞槽重量超过一吨！
每个通道可存储10个91位的字，
差不多1000个脉冲，100通道
@UNIVAC



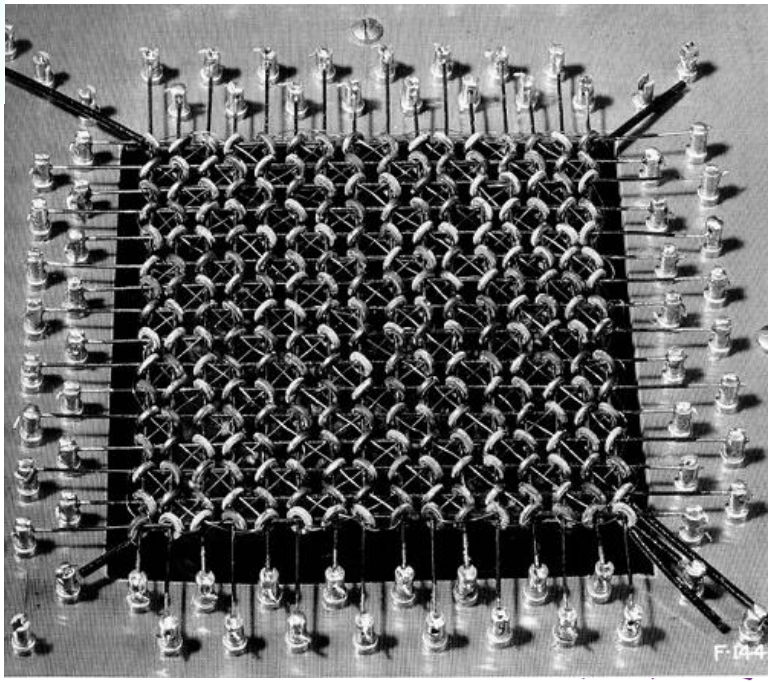
盘式磁带

1950s，IBM最早把盘式磁带用在数据存储上。因为一卷磁带可以代替1万张打孔纸卡

1963，飞利浦
一盘90分钟的录音磁带，在每一面（记得录音磁带是可以翻面的吗）可以存储700KB到1M的数据。

磁鼓和磁芯

1953年，第一台磁鼓应用于IBM701，主存储器；
12英寸长，一分钟可以转1万2千5百转；每支可以保存1万个字符（不到10K）



1948，磁芯存储器
core memory
王安博士
几百个字节
读出是破坏性的，所以读出必须包括一个写入的过程以恢复数据

磁盘

1969, 80K@8’, 1973, 256K@5’

1990s, 1.44M@3.5’

1956, 4.4M@IBM 305 RAMAC硬盘机

2014，4T@WD，￥1399@JD
2015，6T@WD，~2000@JD
2016，8T@WD，~3000@JD

光盘

LD-VCD-DVD

90 000 000

6 000

4 500

0.2%

Holographic Versatile Disc, 3.9TB

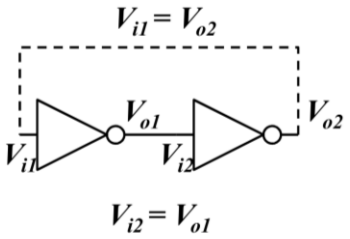
以DVD-R 8.5G容量为标准进行比较

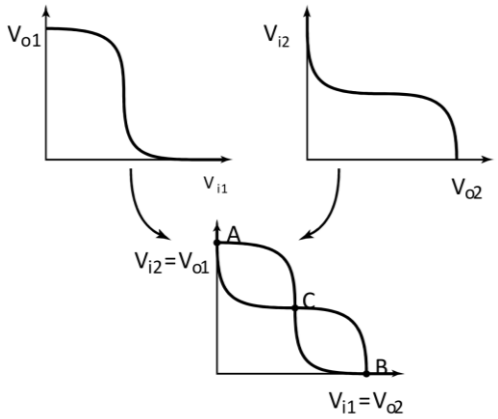
半导体存储器

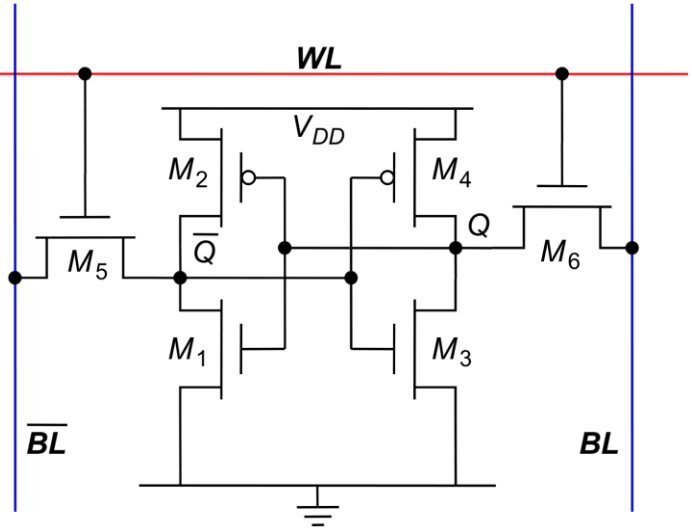
- ROM
  - **Mask-ROM，EPROM，EEPROM，Flash ROM**
- RAM
  - SRAM，DRAM，eDRAM，SDRAM
- Storage Memory
  - Flash, RRAM, PCM, FeRAM, Spintronic

SRAM

- SRAM 存储单元基本结构
  - 两个反相器交叉互连，
    相互钳位形成两个稳定工作点
  - 双稳态工作: 噪声容限大、写入速度高
  - 双位线直接(随机) 访问: 高访问速度
  - 电源电压维持期间数据可长时间保持，
    无需刷新
  - 单元面积大（6T）
  - 用于片上的高速L1, L2, L3 Cache



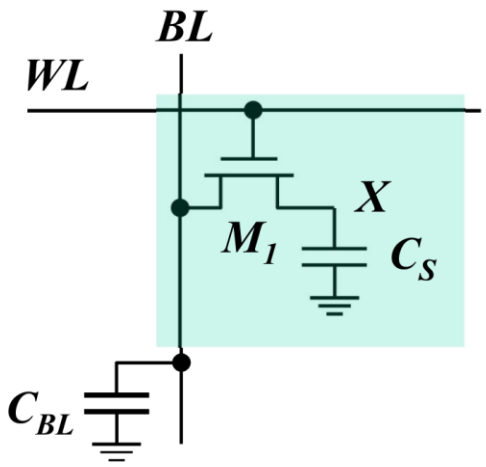


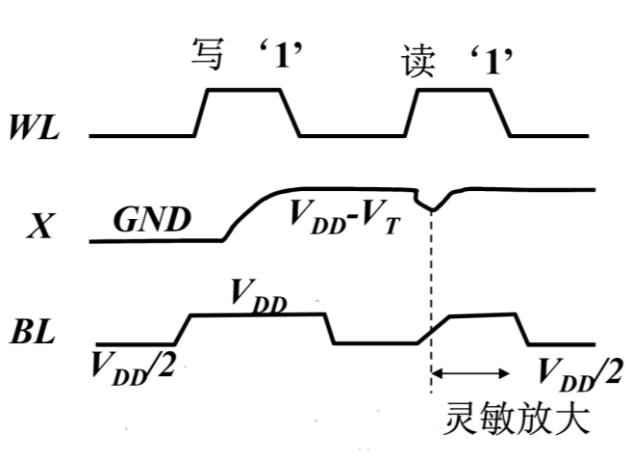


Tsinghua University

DRAM

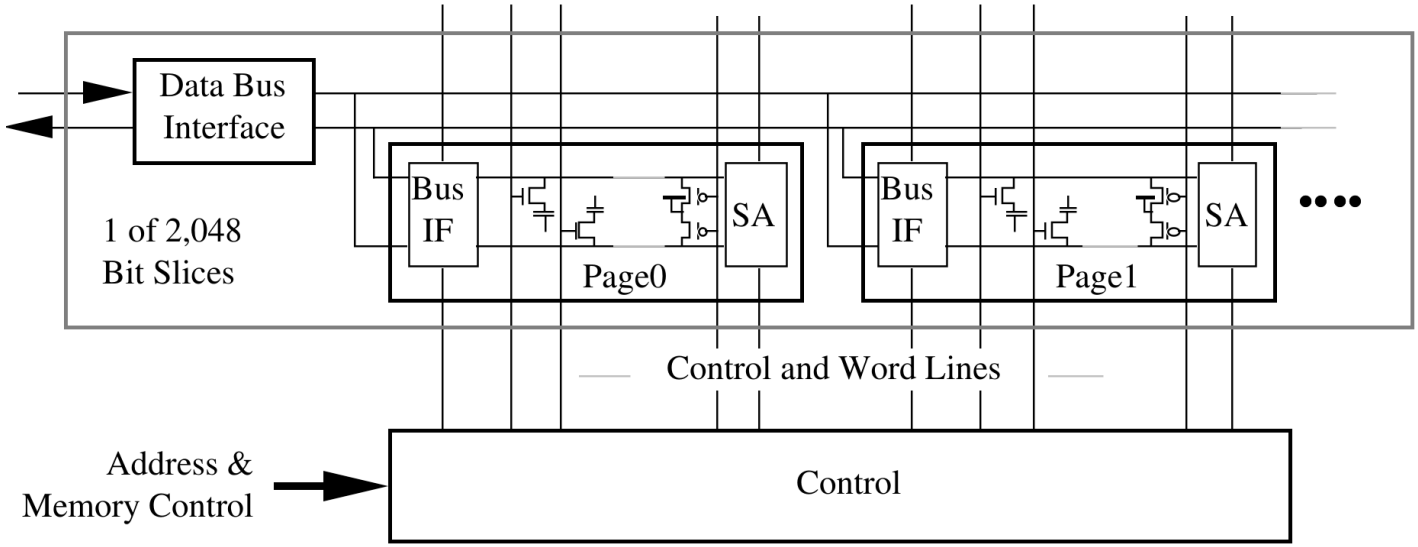
- 1T1C DRAM 存储单元基本结构
  - 1T1C DRAM 的每根位线需要一个灵敏放大器
    （电荷通过C_{BL}在位线上重新分配的读出方式）
  - 单管 DRAM 单元的读出是破坏性的，
    读出时需要进行刷新
  - 单元面积小
  - 速度较慢
  - 通常用于主存储器





eDRAM

- Embedded DRAM
  - 由于1T1C DRAM的高存储密度，被集成在片上使用
  - 与SRAM相比，将DRAM集成在片上需要额外的工艺，因此成本更高
  - 与SRAM相比可节省3x的面积，故可抵消工艺成本
  - 用于L3 甚至L4 Cache
  - 仍需要周期性刷新



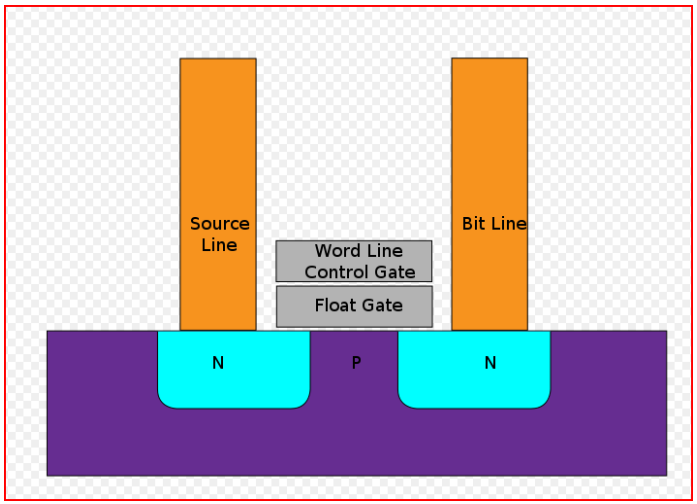
Storage Memory

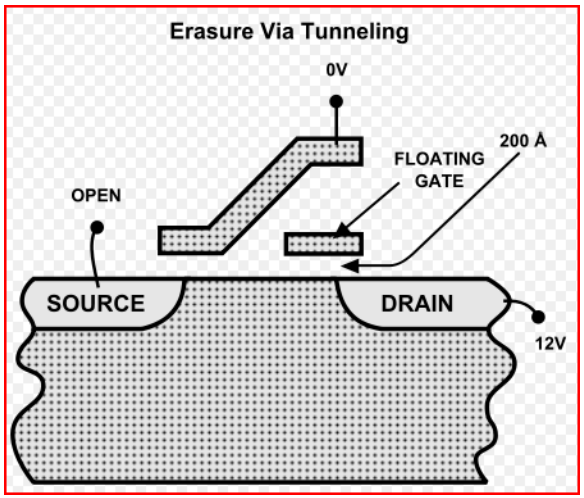
- 非易失存储器 FeRAM, PCM, MRAM, Flash

	单元面积（um^2）	编程电压 (V)	编程速度 (ns)	擦写次数	抗辐射	读写功耗	应用领域
Flash	0.014	12-15	1000	1E+5	弱	大	通用
FeRAM	0.121	0.9-3.3	30-35	1E+15	强	小	低功耗IC
MRAM	0.07	0.9-3.3	<30	无限次	强	中	高性能IC
PCM	N.A.	3	<100	1E+15	强	大	大容量存储

Flash issues

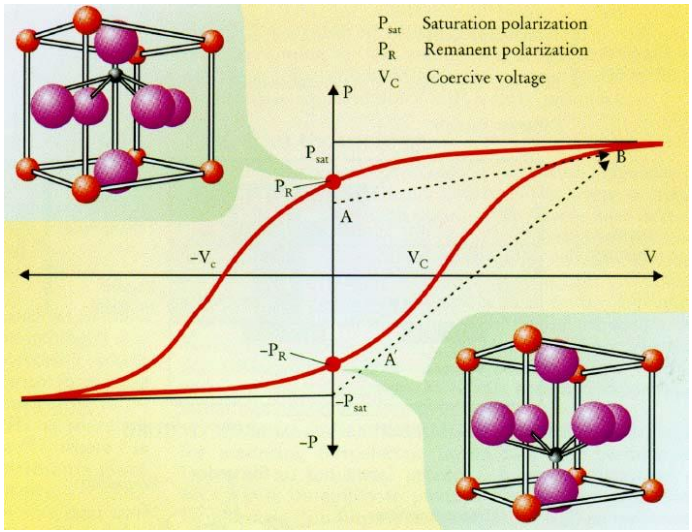
- Flash is programmed at system voltages.
- Erasure time is long.
- Must be erased in blocks.





FeRAM - Theory

Binary state 0
Positive electric field
Positive polarization

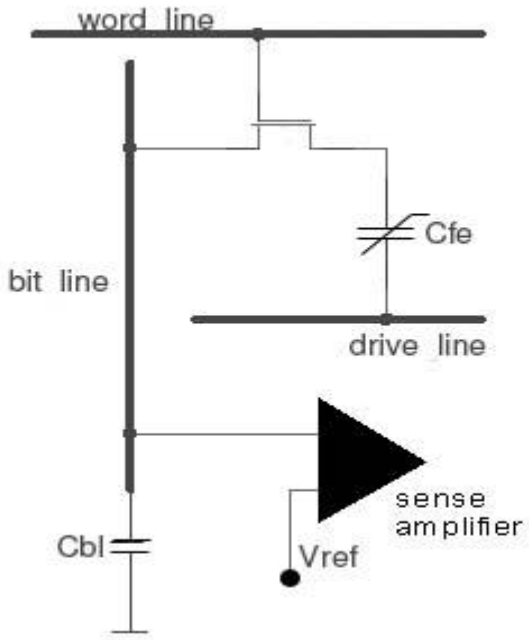


Example: PZT
(lead zirconate-titanate)

Binary state 1
Negative electric field
Negative polarization

- Spontaneous polarization: above the Curie-temperature T_C is the structure cubic, below a dipole moment occurs (displacement)
- A different charge ΔQ can be observed whether the material is switching or non-switching:

FeRAM - 1T/1C-Cell

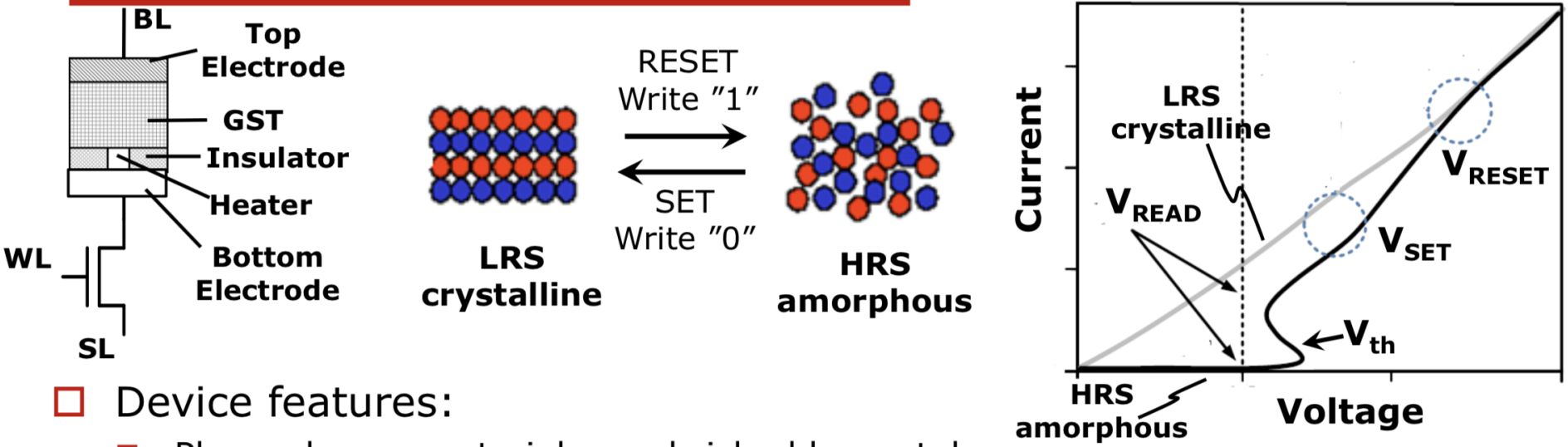


- Write
  - WL: adressed
  - DL: pulse $+V_{CC}$ (half length)
  - BL: $+V_{CC}$: “1”, ground:”0”

- Read
  - WL: adressed
  - DL: adressed with positive voltage $+V_{cc}$
  - BL: capacitor divider between C_{fe} and C_{bl}, sense amplifier compares voltage with V_{ref}
    - $V<V_{ref}$: Binary state 0
    - $V>V_{ref}$: Binary state 1
  - But: reading operation is destructive, information needs to be restored

PCM

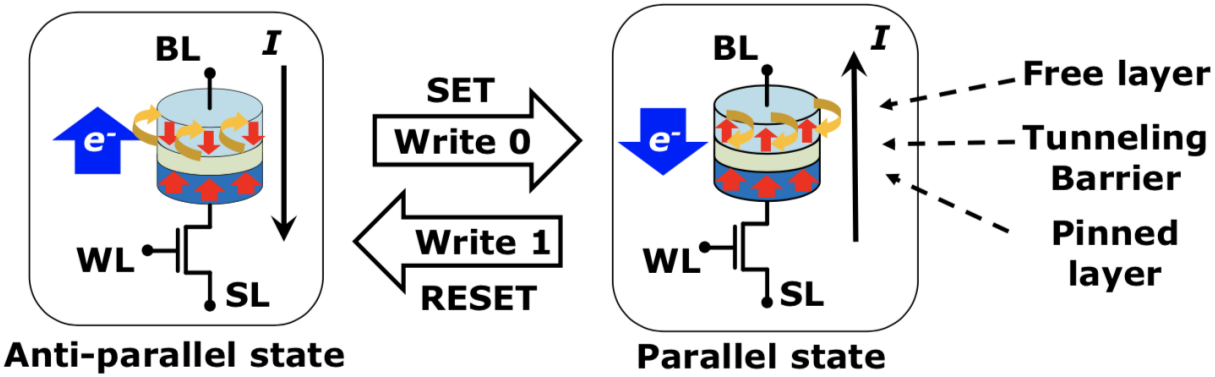
Phase Change Memory (PCM/PRAM)

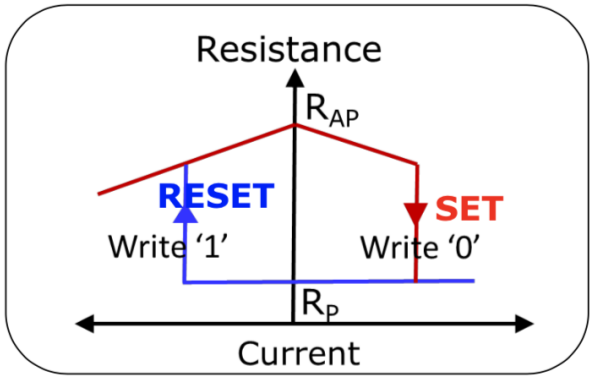


- Device features:
  - Phase change materials sandwiched by metals
    Confined bottom electrode as "heater"
  - Material: $Ge_2Sb_2Te_5$ (GST)
  - LRS: Crystalline state (low-R); HRS: Amorphous state (high-R)
  - Thermal-based memory device (joule heating – current controlled)

MRAM

Spin Transfer Torque Magnetic Memory (STT-MRAM)





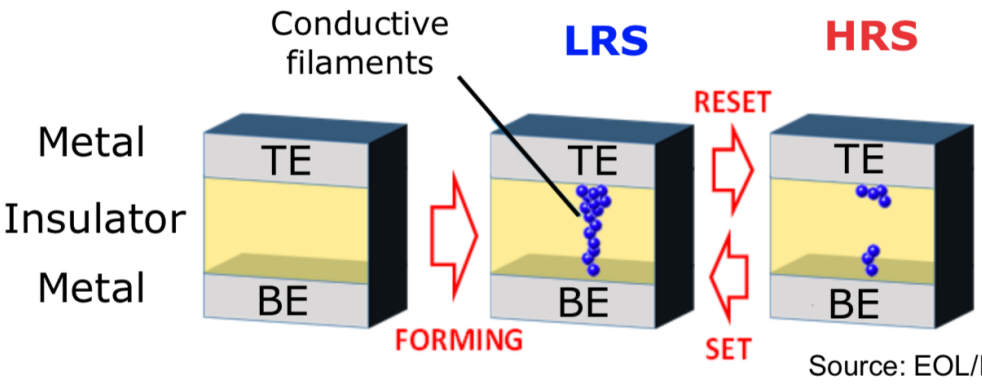
- Devices features:
  - Magnetic tunnel junction (MTJ)
  - Ferromagnetic layers and tunnel barrier (typical: MgO)
  - Parallel ("0"): low-R/LRS (R_P); Anti-Parallel ("1"): high-R/HRS (R_{AP})
- Write by spin-polarized current

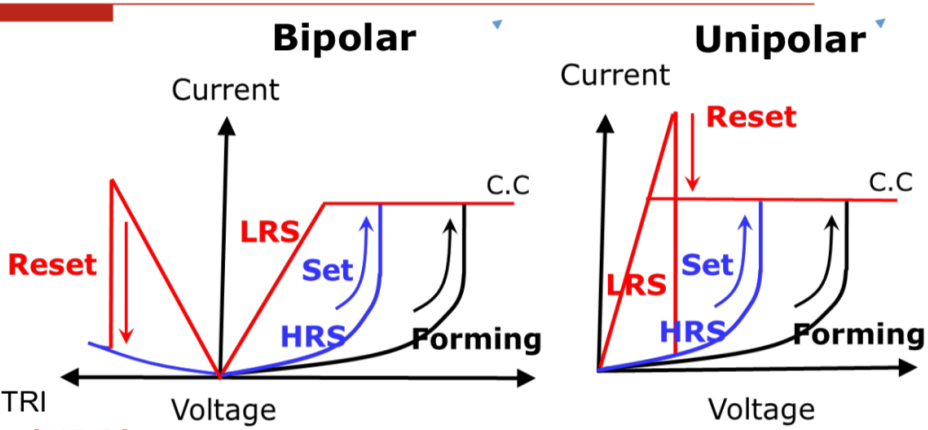
1T+1R cell bias table

Operation	SET	RESET	Read
WL	V_{G_SET}	VDD	VDD
BL	V_{SET}	0	V_{READ}
SL	0	V_{RESET}	0
State	LRS, R_P (0)	HRS, R_{AP} (1)	(0/1)

ReRAM

Introduction of RRAM/ReRAM – Write Operations





Low-Resistive-State (LRS) vs. High-Resistive-State (HRS)

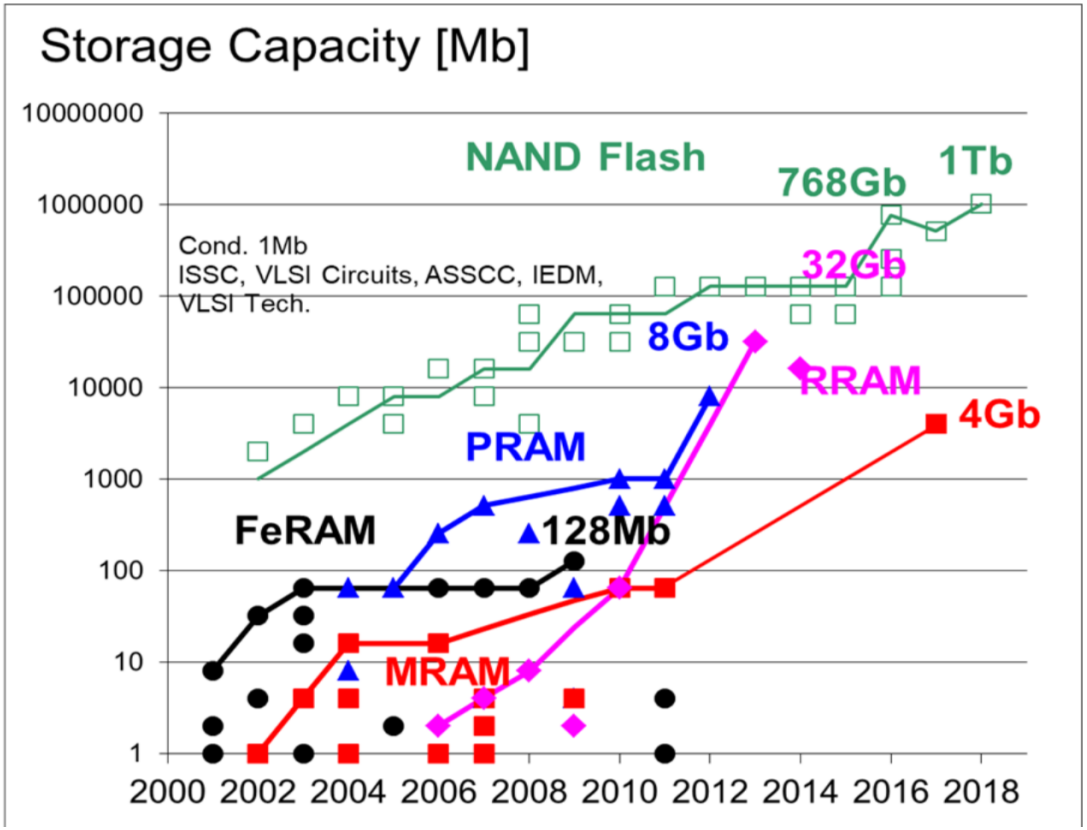
- Write Operations
  - Forming: activation (one time only)
  - SET (write-0): from HRS to LRS
  - RESET (write-1): from LRS to HRS
  - Bipolar and unipolar types

1T+1R bipolar cell bias table

	Forming	SET	RESET	Read
WL	V_{G_SET}	V_{G_SET}	VDD	VDD
BL	$V_{Forming}$	V_{SET}	0	V_{READ}
SL	0	0	V_{RESET}	0
State	LRS (0)	LRS (0)	HRS (1)	0/1

ISSCC 2018 Technology Trend

Recent NVM Development- Storage Capacity



- NAND achieves highest density
- Max. storage capacity of emerging NVM increases significantly

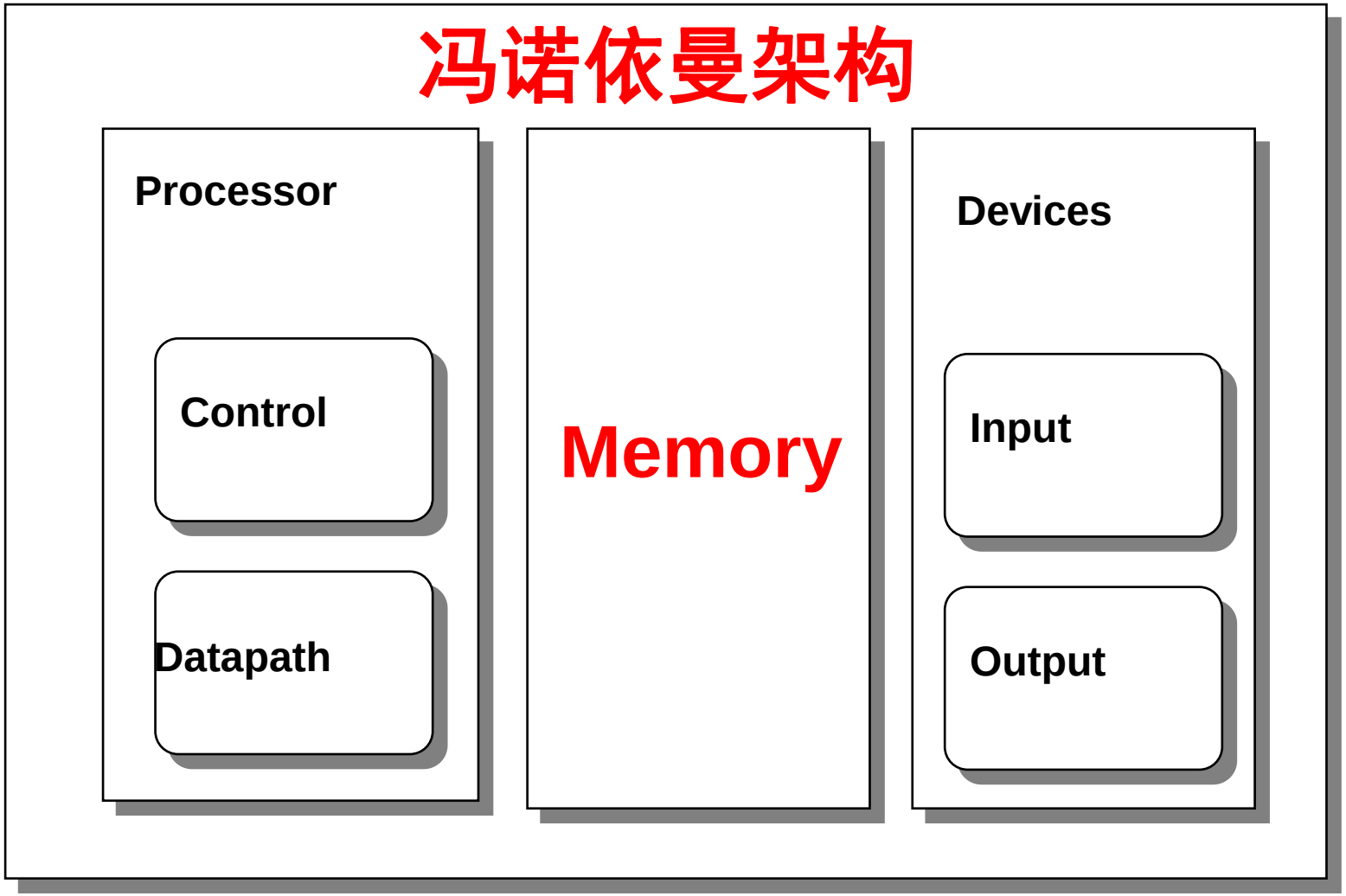
[Source: ISSCC 2018 Trend]

计算机系统存储器现状对比

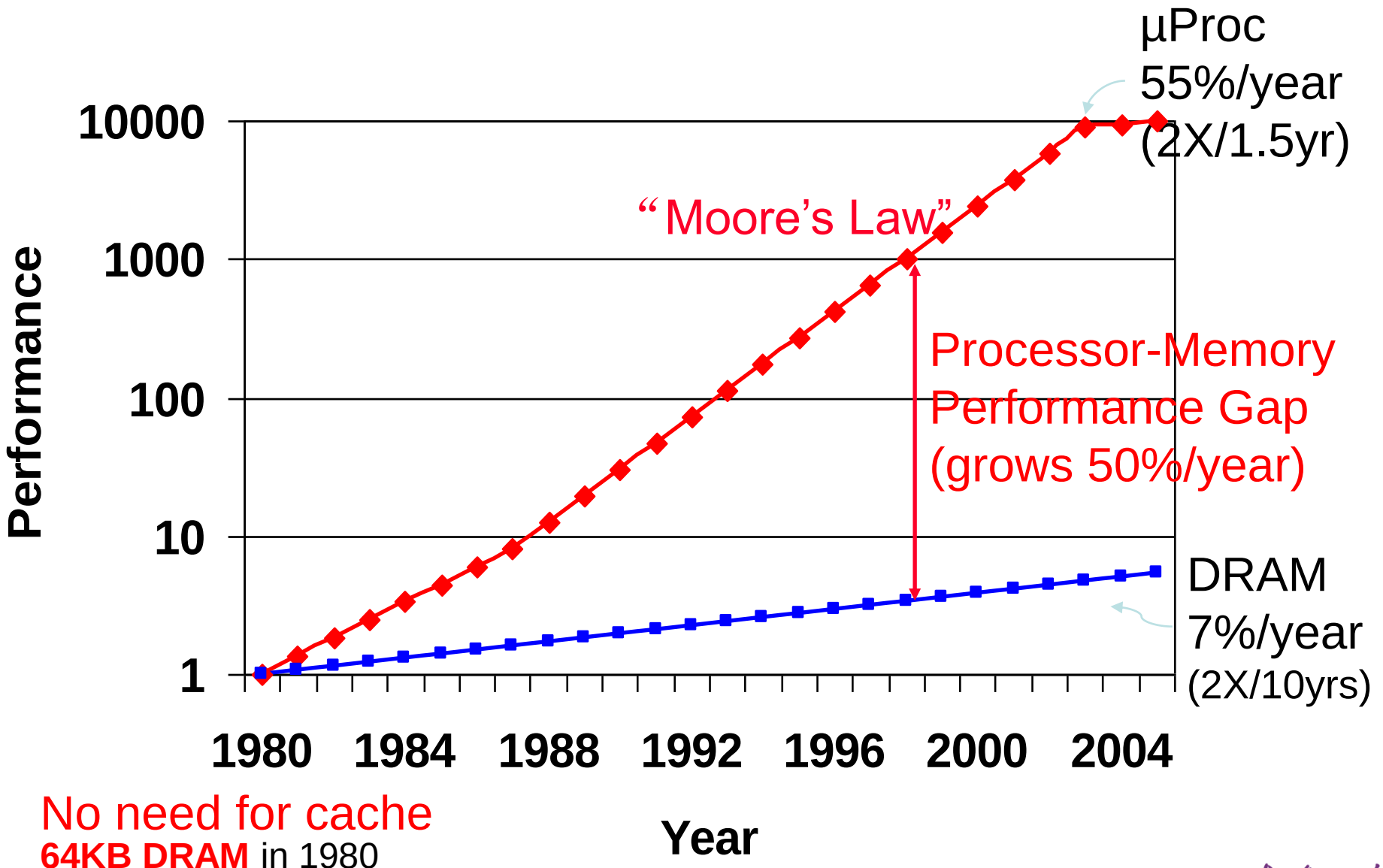
- 存储设备进化几十年与PC的发展
  - 来源： 中文业界资讯站　发布时间： 2014年03月24日

- 提升容量
- 提升读写速度
- 降低成本
- 非易失性

Major Components of a Computer



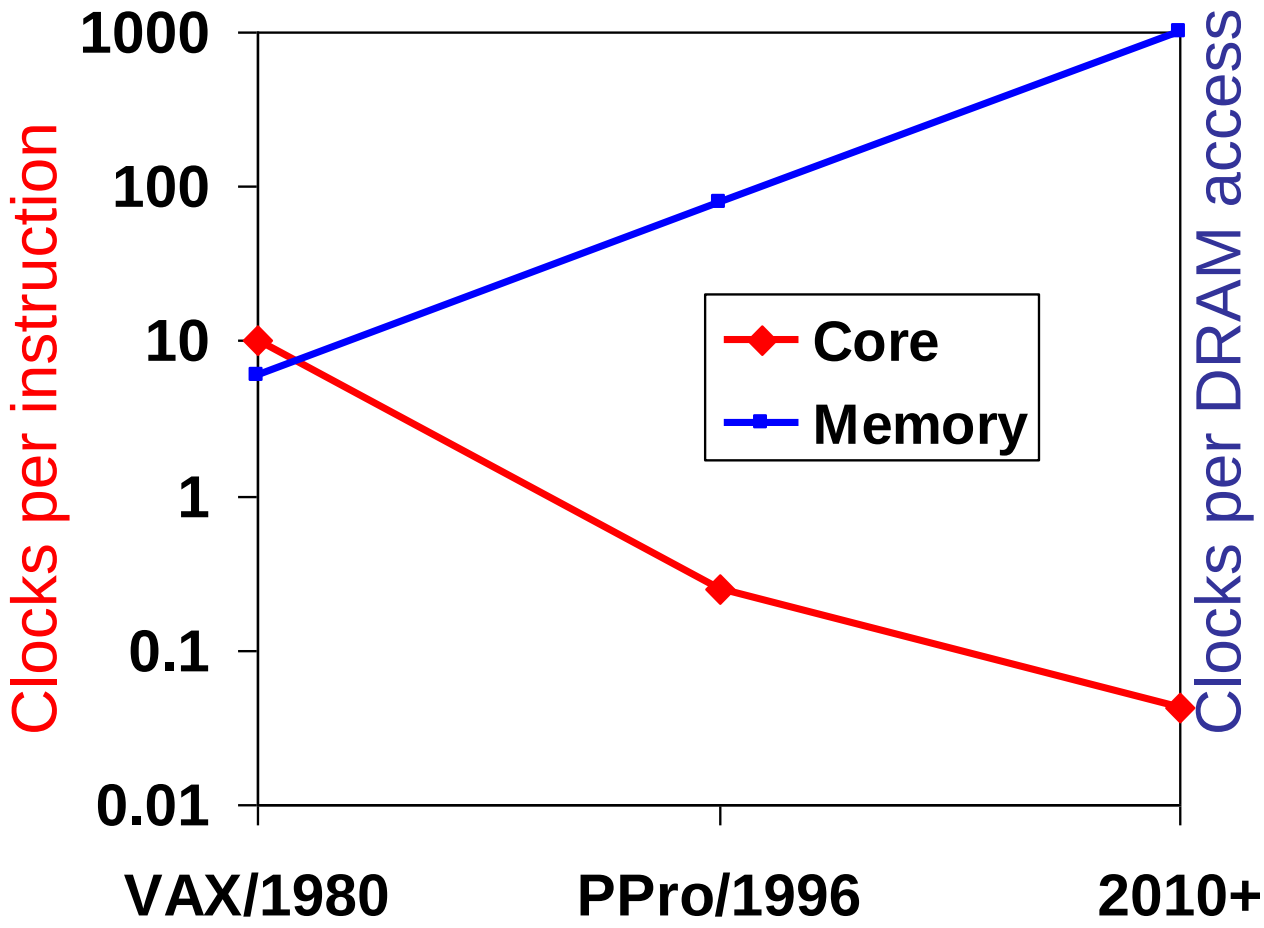
Processor-Memory Performance Gap



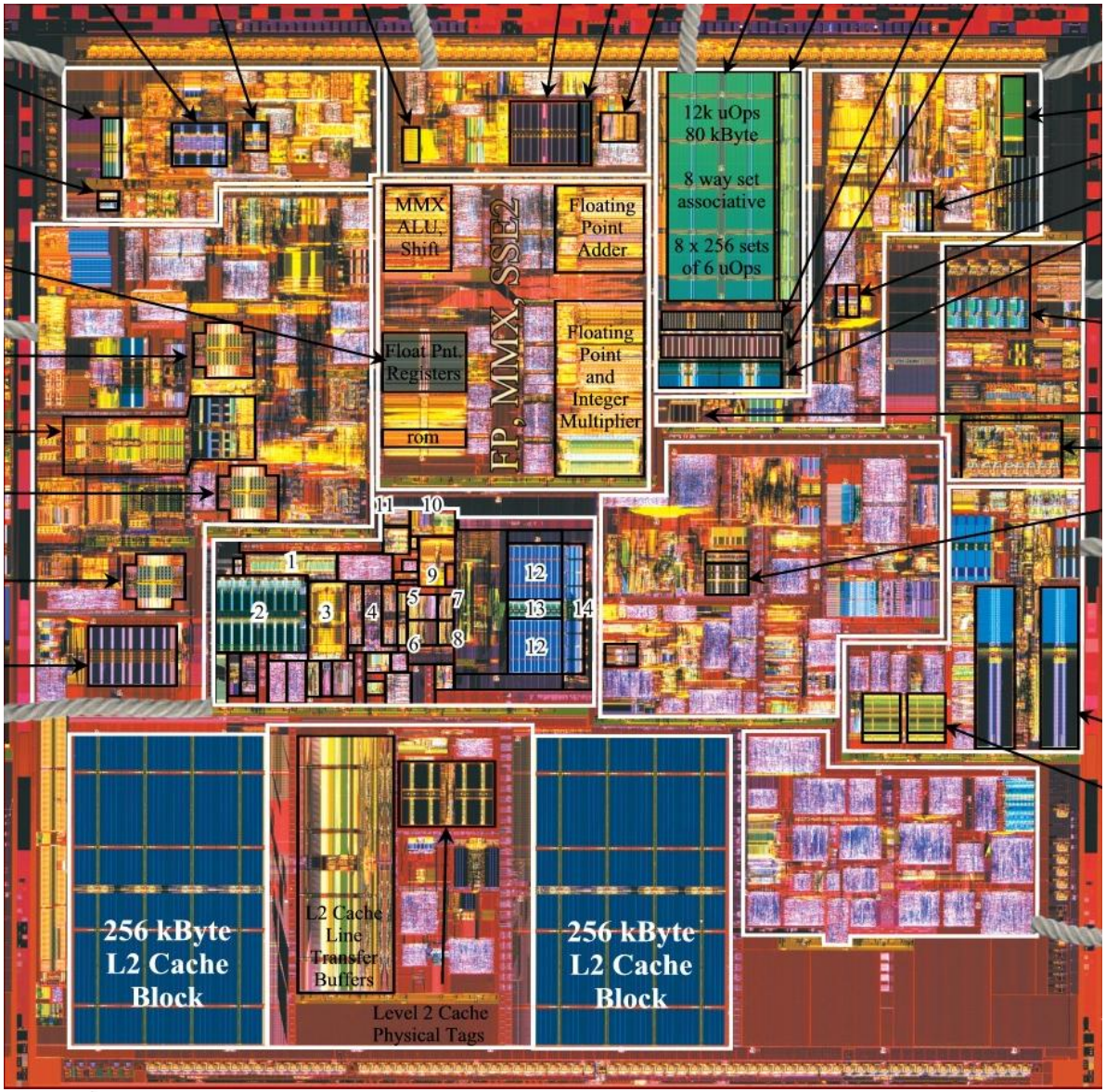
No need for cache
64KB DRAM in 1980

The “Memory Wall”

- Logic vs. DRAM speed gap continues to grow



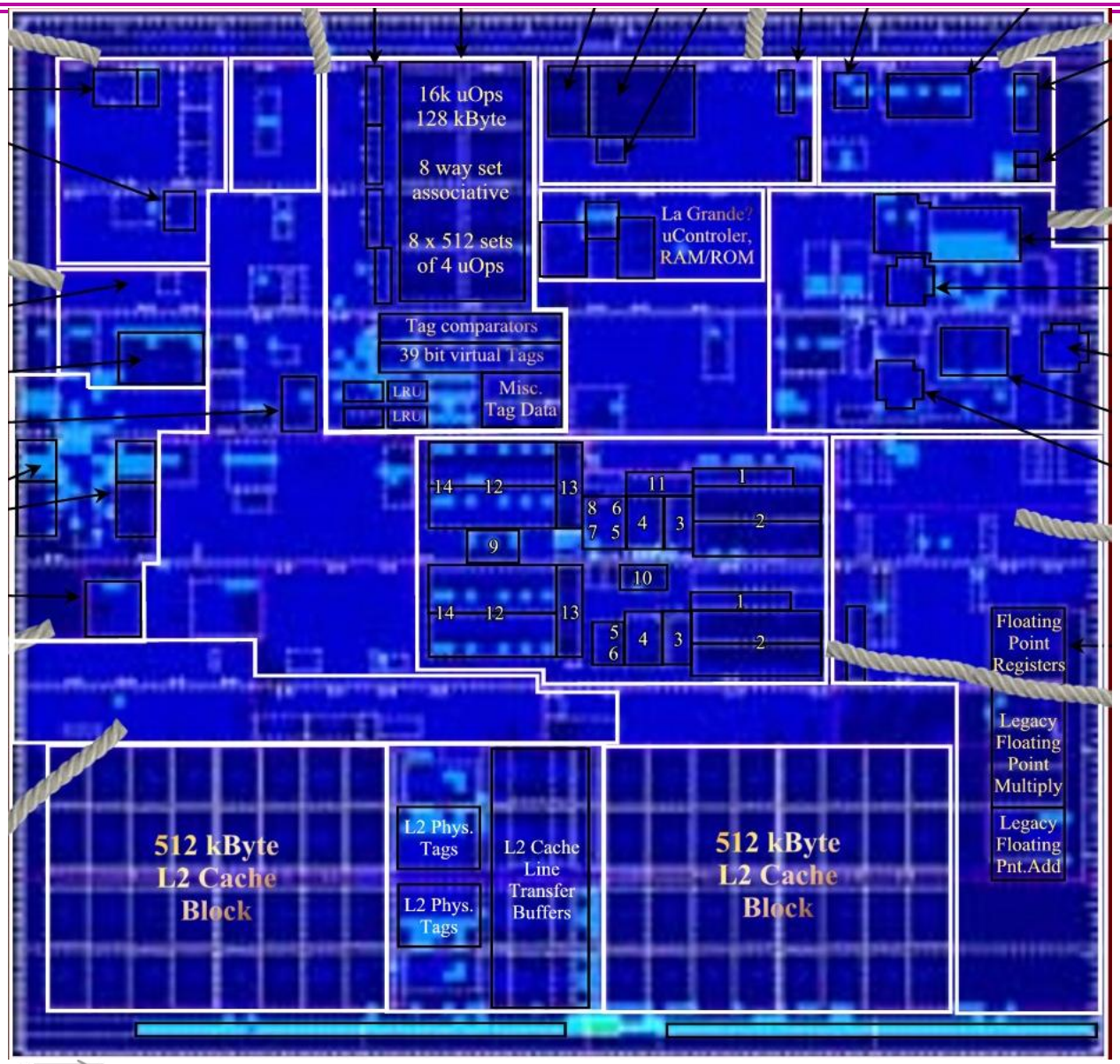
Intel Pentium 4—Northwood (2002)



- 0.13 um
- Area: 146 mm2
- 55 M transistors
- L1-Instruction: 12K
- L1-Data: 8K
- L2: 512K

Tsinghua University

Intel Pentium 4 –Prescott (2004)



- 90 nm
- Area: 112 mm2
- 125 M transistors
- L1-Instruction: 16K
- L1-Data: 16K
- L2: 1MB

Question:
a. Why L1 cache is divided?
b. Why L1 is small?
c. Why hierarchy?

Memory Hierarchy: Goals

- Fact: Large memories are slow, fast memories are small
- How do we create a memory that gives the illusion of being large, cheap and fast (most of the time)?

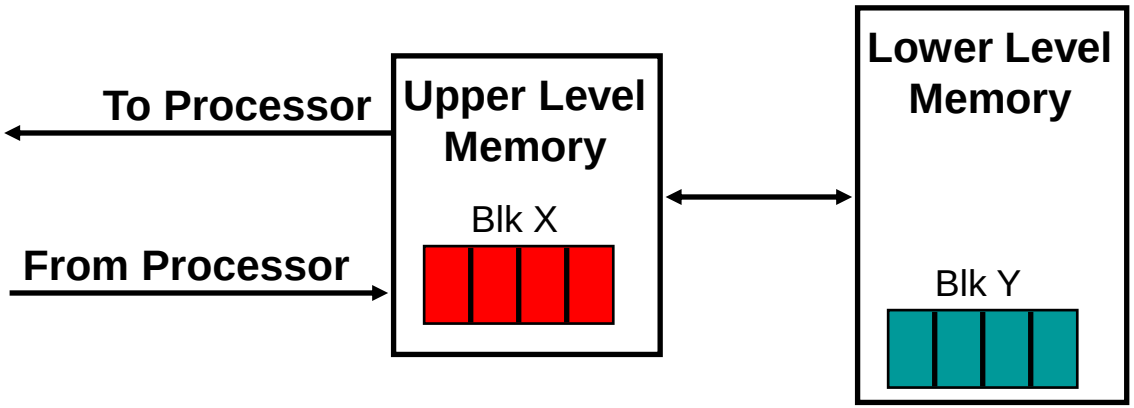
- The Principle of locality states : Programs access a relatively small portion of the address space at any instant of time.

Memory Hierarchy: Why Does it Work?

- Temporal Locality (Locality in Time):
- 例如运算常量，卷积模板

  => Keep most recently accessed data items closer to the processor
- Spatial Locality (Locality in Space):
- 例如图像和视频

  => Move blocks consists of contiguous words (adjacent)to the upper levels



You are using this “caching” idea already!

- Temporal Locality (Locality in Time):
  - => Keep most recently accessed data items closer to the processor
- Spatial Locality (Locality in Space):
  - => Move blocks consists of contiguous words to the upper levels

Temporal

Spatial

Reg. / TLB

MM

Cache

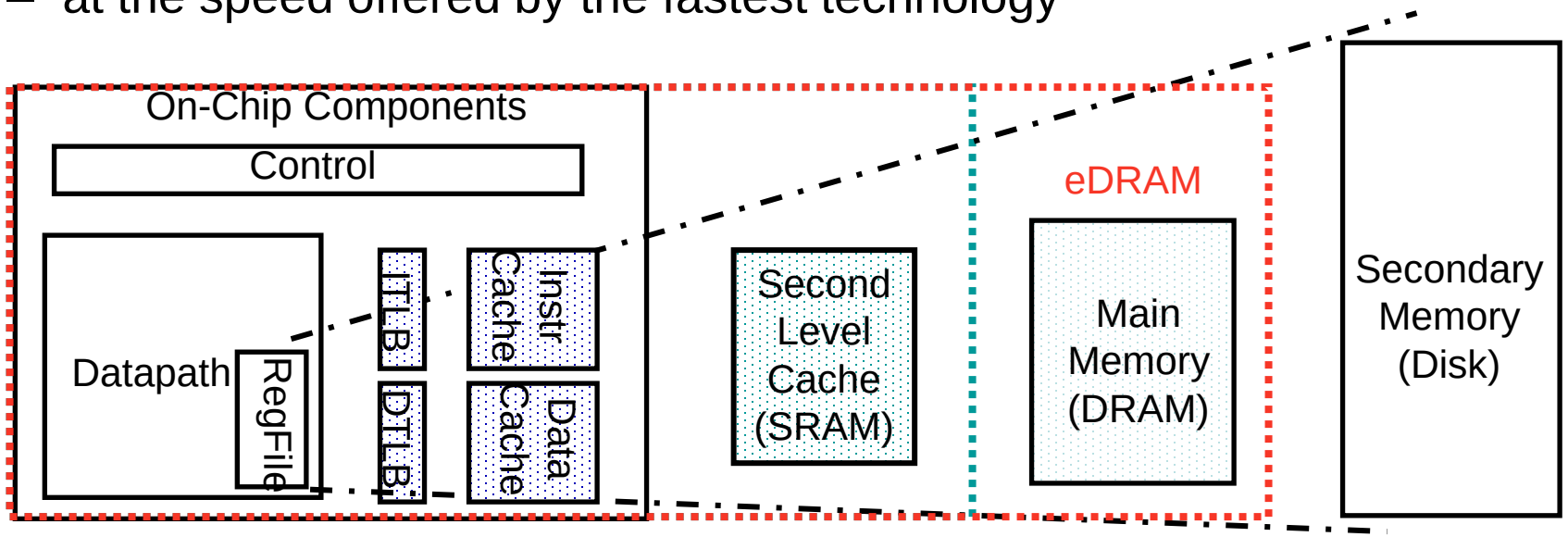
Memory Hierarchy Technologies

- **Random Access**
  - "Random" is good: access time is the same for all locations
  - _DRAM_: Dynamic Random Access Memory
    - High density (1 transistor cells), low power, cheap, slow
    - Dynamic: need to be "refreshed" regularly (~ every 8 ms)
  - _SRAM_: Static Random Access Memory
    - Low density (6 transistor cells), high power, expensive, fast
    - Static: content will last "forever" (until power turned off)
  - Size: SRAM/DRAM *4 to 8, even bigger*
  - Cost/Cycle time: SRAM/DRAM *8 to 16*

- **"Non-so-random" Access Technology**
  - Access time varies from location to location and from time to time (e.g., Disk, CDROM)

A Typical Memory Hierarchy

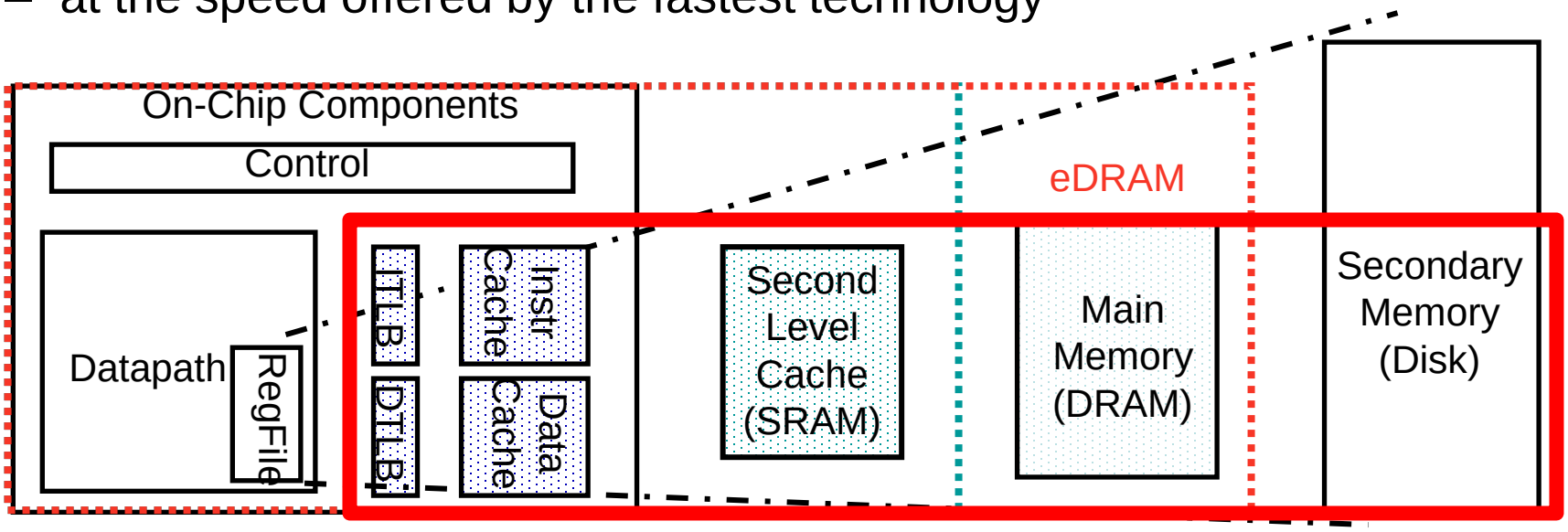
- By taking advantage of the principle of locality
  - Can present the user with as much memory as is available in the cheapest technology
  - at the speed offered by the fastest technology



Speed (%cycles):	½'s	1's	10's	100's	1,000's
Size (bytes):	100's	K's	10K's	M's	G's to T's
Cost:	highest				lowest

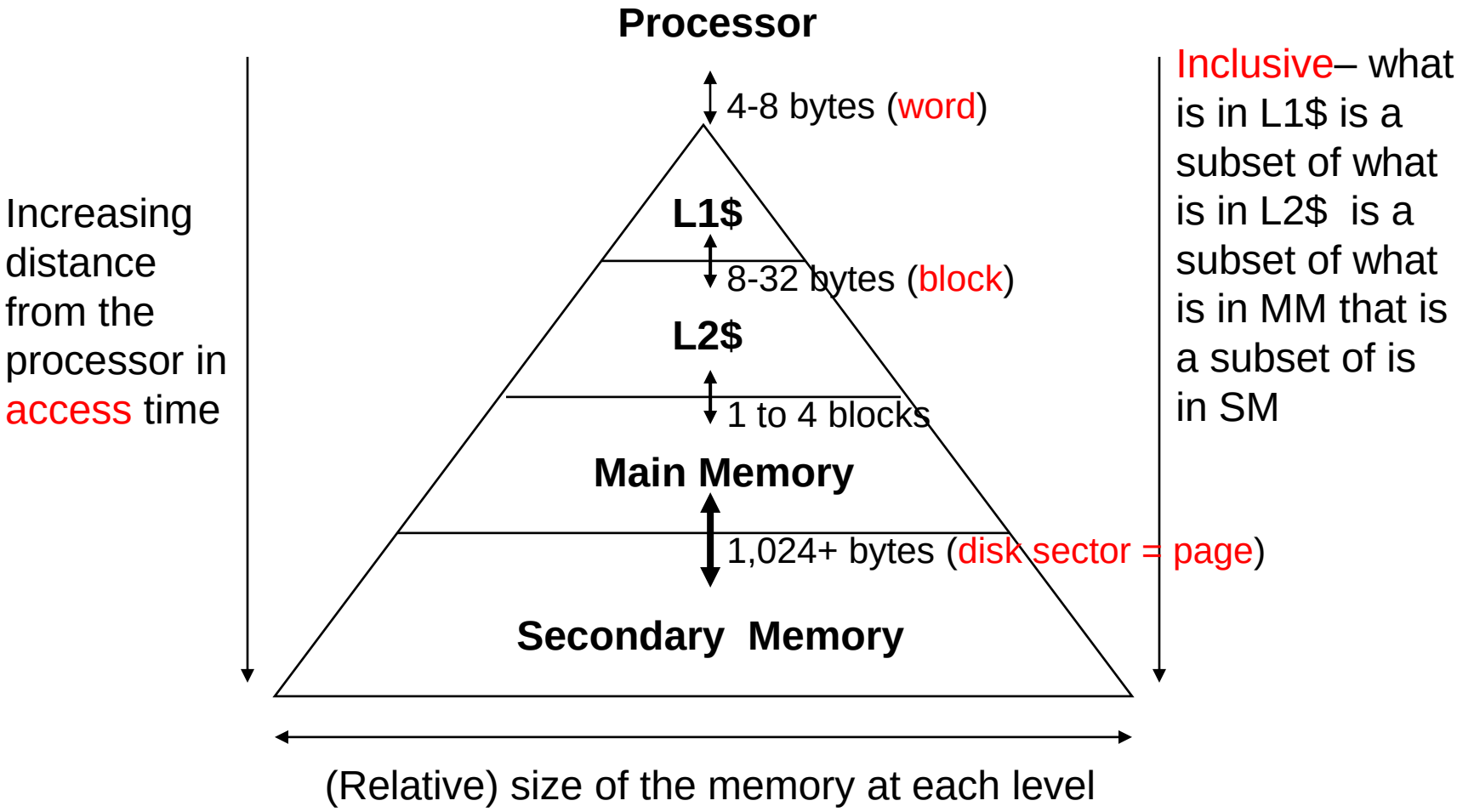
A Typical Memory Hierarchy

- By taking advantage of the principle of locality
  - Can present the user with as much memory as is available in the cheapest technology
  - at the speed offered by the fastest technology



Speed (%cycles):	½'s	1's	10's	100's	1,000's
Size (bytes):	100's	K's	10K's	M's	G's to T's
Cost:	highest				lowest

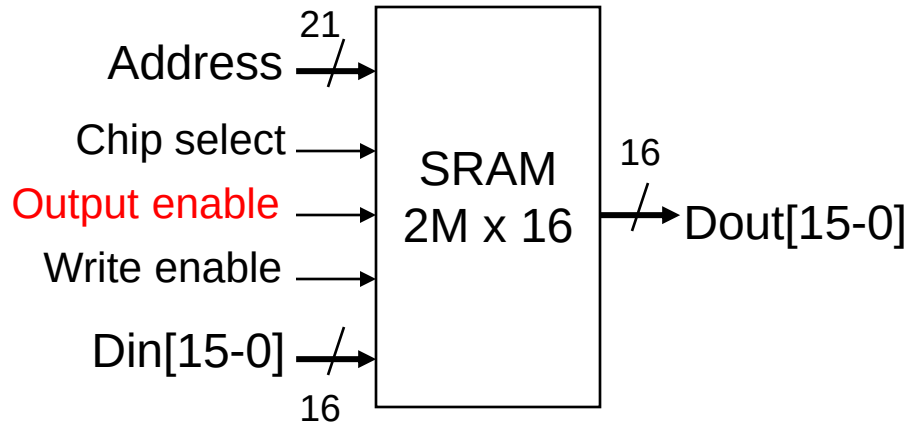
Characteristics of the Memory Hierarchy



Tsinghua University

Memory Hierarchy Technologies

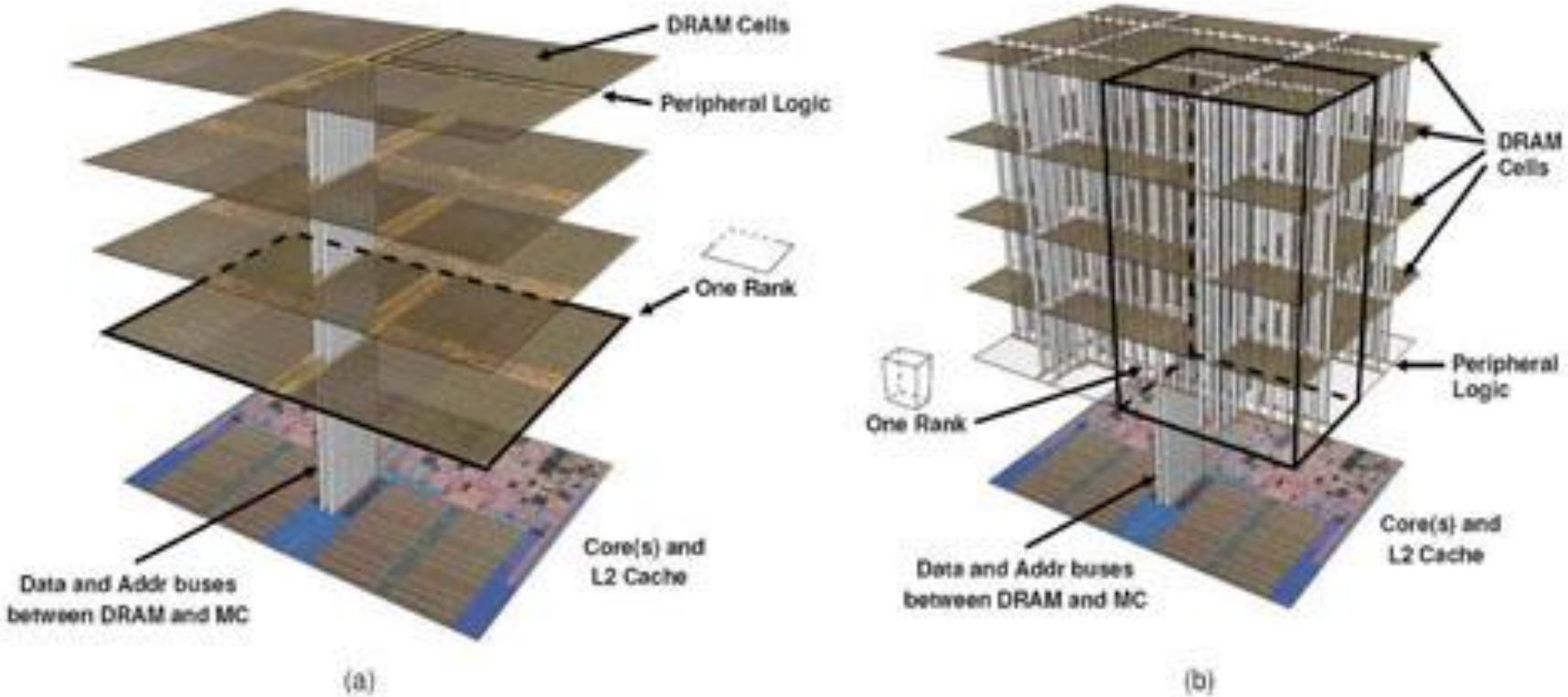
- Caches use *SRAM* for speed and technology compatibility
  - Low density (6 transistor cells), high power, expensive, fast
  - Static: content will last "forever" (until power turned off)



- Main Memory uses *DRAM* for size (density)
  - High density (1 transistor cells), low power, cheap, slow
  - Dynamic: needs to be "refreshed" regularly (~ every 8 ms)
    - 1% to 2% of the active cycles of the DRAM
  - Addresses divided into 2 halves (row and column)
    - RAS or Row Access Strobe triggering row decoder
    - CAS or Column Access Strobe triggering column selector

Memory Hierarchy Technologies

- Caches use *DRAM* with 3D *TSV*



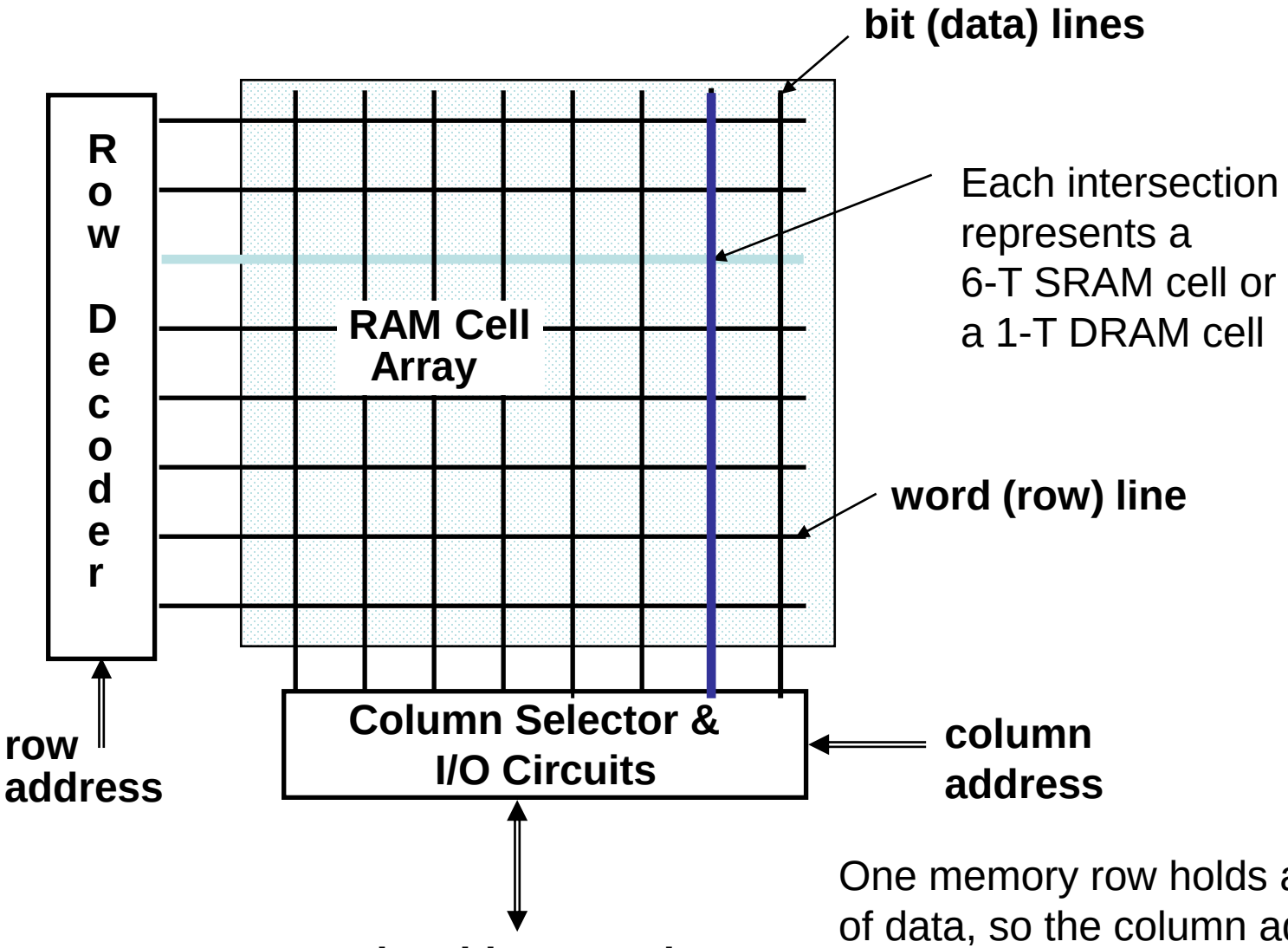
C. C. Liu
@Cornell University, 2005

Tsinghua University

Memory Performance Metrics

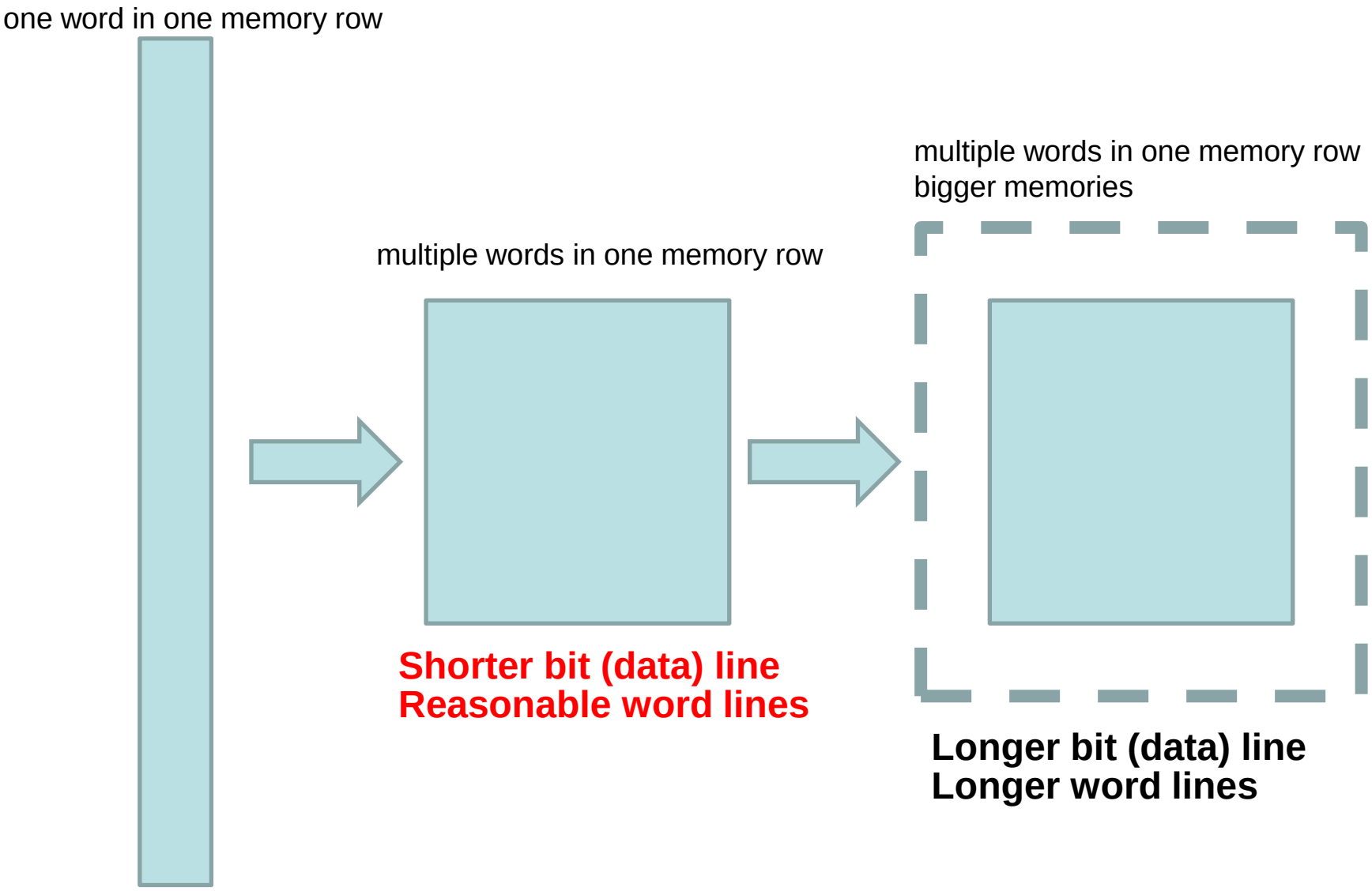
- Latency: Time to access one word
  - *Access time*: time between the request and when the data is available (or written)
  - *Cycle time*: time between requests
  - Usually cycle time > access time
  - Typical read access times for SRAMs in 2004
    - 2 to 4 ns for the fastest parts
    - 8 to 20ns for the typical largest parts
- Bandwidth: How much data from the memory can be supplied to the processor per unit time
  - width of the data channel * the rate at which it can be used
- *Size*: SRAM to DRAM *4 to 8*
- *Cost/Cycle time*: SRAM to DRAM *8 to 16*

Classical RAM Organization (~Square)

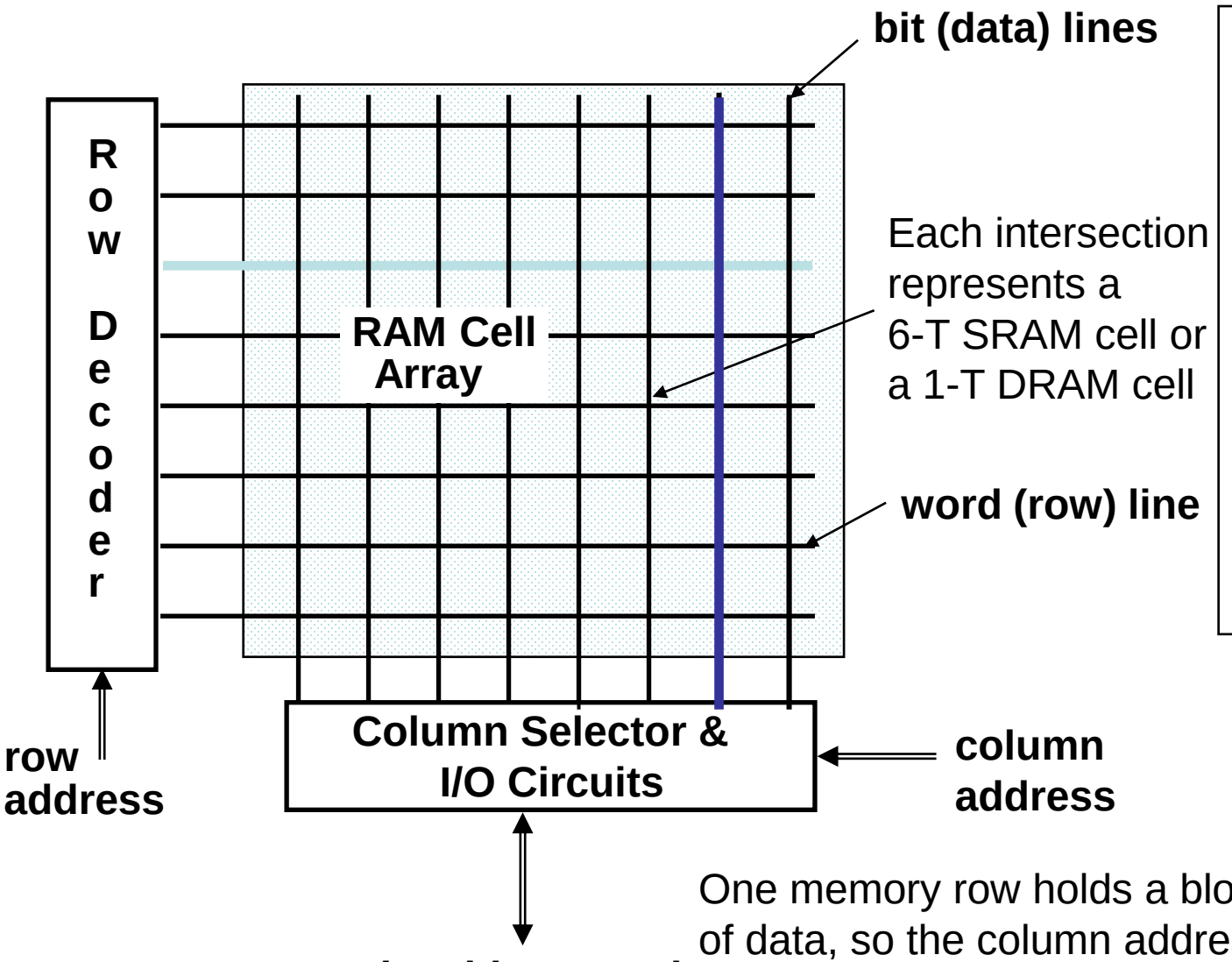


One memory row holds a block of data, so the column address selects the requested bit or word from that block

A2:Classical RAM Organization



A2:Classical RAM Organization (~Square)



e.g.
Column address
64 words / block(row)
6bits
32 bits / word
5bits

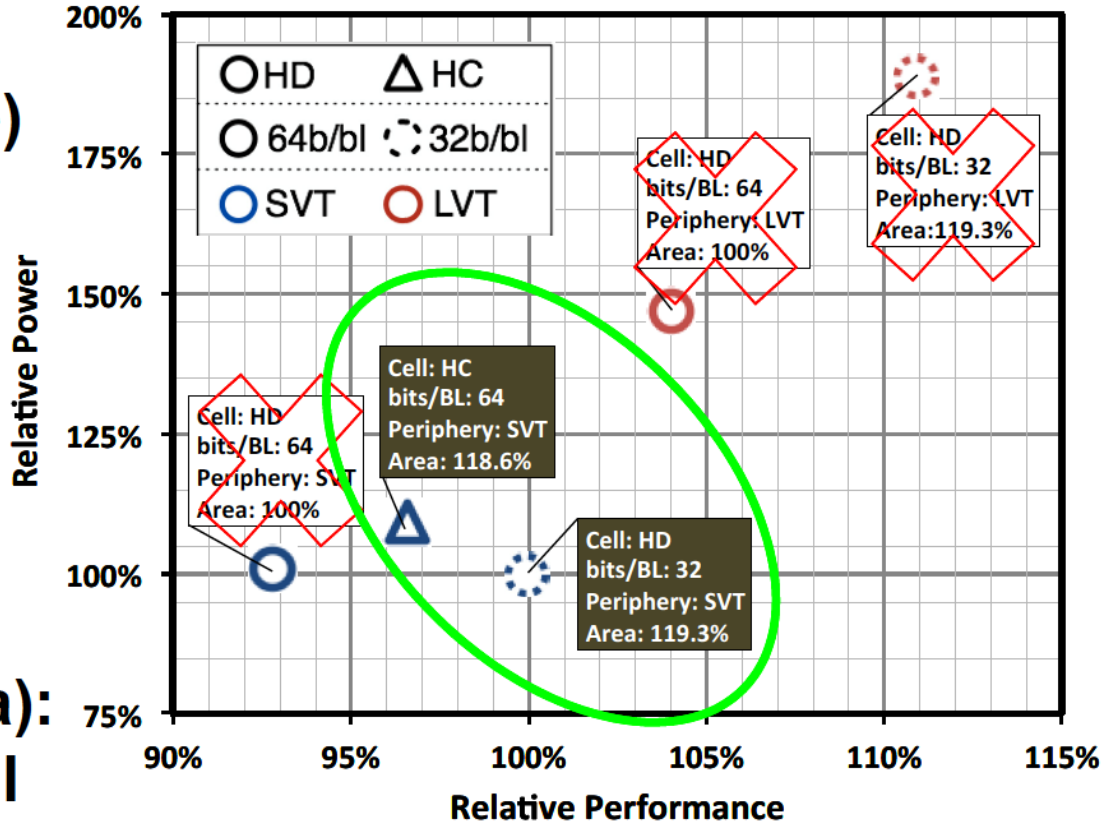
Row address
7bits

18bits for RAM Cell Array of 256Kb

One memory row holds a block of data, so the column address selects the requested bit or word from that block

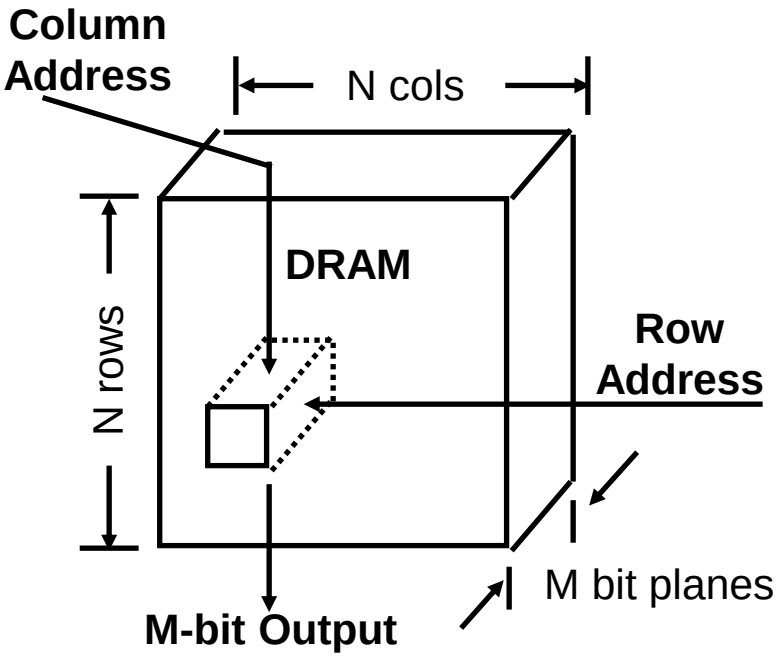
L1 SRAMs PPA Trade-off

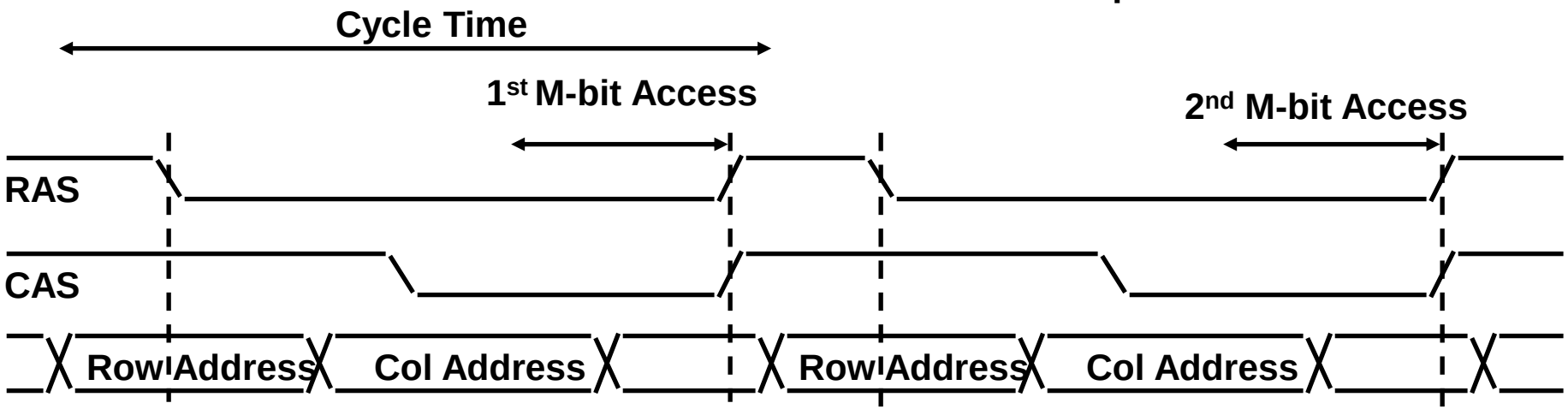
- 3 key design options were explored for the L1 cache
  1. High-Density (HD) vs. High-Current (HC) bit-cell
  2. 64 bits/bitline (b/bl) vs. 32b/bl
  3. SVT vs. LVT periphery transistors
- Limit to +25% Area (HC-32b/bl too large)
- LVT eliminated due to leakage power and mask cost
- HD-64b/bl did not provide enough performance
- Compare (similar area): HD-32b/bl & HC-64b/bl



Classical DRAM Operation

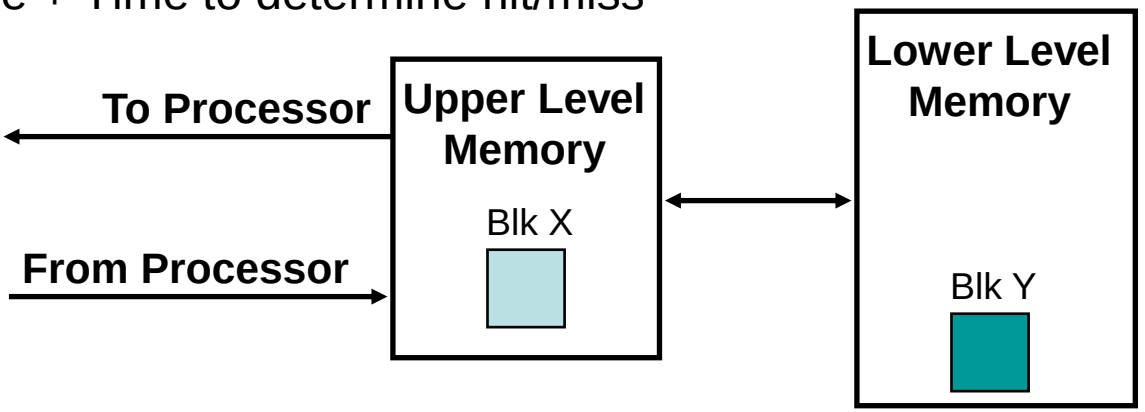
- DRAM Organization:
  - N rows x N column x M-bit
  - Read or Write M-bit at a time
  - Each M-bit access requires a RAS / CAS cycle





Memory Hierarchy: Terminology

- Hit: data appears in some block in the upper level (Block X)
  - Hit Rate: the fraction of memory accesses found in the upper level
  - Hit Time: Time to access the upper level which consists of
    RAM access time + Time to determine hit/miss

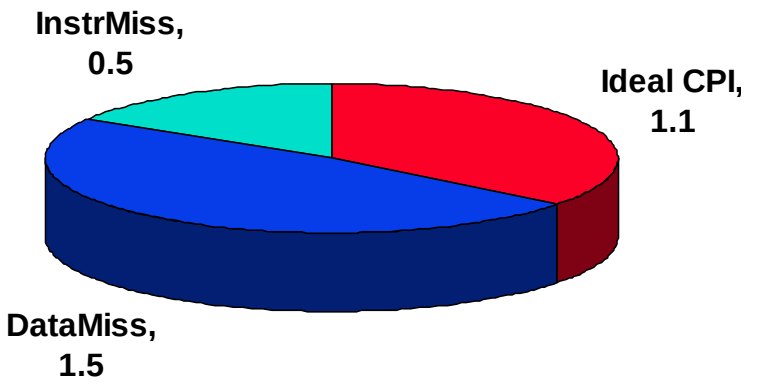


- Miss: data needs to be retrieve from a block in the lower level (Block Y)
  - Miss Rate = 1 - (Hit Rate)
  - Miss Penalty: Time to replace a block in the upper level
    + Time to deliver the block the processor
  - Hit Time << Miss Penalty

Memory Performance Impact on Performance

- Suppose a processor executes at
  - ideal CPI = 1.1
  - 50% arith/logic, 30% ld/st, 20% control

  and that 10% of data memory operations miss with a 50 cycle miss penalty



- CPI = ideal CPI + average stalls per instruction

  = 1.1(cycle) + (0.30 (datamemops/instr)
  x 0.10 (miss/datamemop) x 50 (cycle/miss))

  = 1.1 cycle + 1.5 cycle

  = 2.6

  so 58% of the time the processor is stalled waiting for memory!

- A 1% instruction miss rate would add an additional 0.5 to the CPI!
  - 0.01*50 = 0.5

Summary of these Slides

- DRAM is slow but cheap and dense
  - Good choice for presenting the user with a BIG memory system
- SRAM is fast but expensive and not very dense
  - Good choice for providing the user FAST access time
- Two different types of locality
  - Temporal Locality (Locality in Time): If an item is referenced, it will tend to be referenced again soon.
  - Spatial Locality (Locality in Space): If an item is referenced, items whose addresses are close by tend to be referenced soon.
- By taking advantage of the principle of locality:
  - Present the user with as much memory as is available in the cheapest technology.
  - Provide access at the speed offered by the fastest technology.